Experiment No. 2(a) Date:

Experiment Question

For a given set of training data examples stored in a .CSV file, implement and demonstrate the Candidate-Elimination algorithm in python to output a description of the set of all hypotheses consistent with the training examples. [Sub-program (a): EnjoySport data set, using pandas]

Aim

To implement the Candidate-Elimination algorithm in Python using pandas, reading the training examples of the EnjoySport data set, and to output the specific boundary S and the general boundary G that describe all the hypotheses consistent with the training examples.

Concept

The version space is the subset of hypotheses of H that are consistent with all the training examples. The Candidate-Elimination algorithm represents it by two boundaries: S, the set of maximally specific hypotheses, and G, the set of maximally general hypotheses. Every hypothesis lying between S and G is consistent with the data.

A positive example generalises S and removes the inconsistent members of G. A negative example specialises G and removes the inconsistent members of S. Unlike FIND-S, the algorithm uses both positive and negative examples.

Dataset Description

The EnjoySport data set (enjoysport.csv) has 4 training examples, 6 input attributes (Sky, Temp, Humidity, Wind, Water, Forecast) and the target attribute EnjoySport (Yes / No).

Sky	Temp	Humidity	Wind	Water	Forecast	EnjoySport
Sunny	Warm	Normal	Strong	Warm	Same	Yes
Sunny	Warm	High	Strong	Warm	Same	Yes
Rainy	Cold	High	Strong	Warm	Change	No
Sunny	Warm	High	Strong	Cool	Change	Yes

Algorithm

1. Initialise S to the first (positive) training example and G to the most general hypothesis <?, ?, ..., ?>.
2. For each training example d:
   - If d is a positive example: remove from G every hypothesis inconsistent with d, and generalise S (replace by "?" every attribute that differs from d).
   - If d is a negative example: every member g of G that covers d is replaced by its minimal specialisations that exclude d and are still more general than S; then keep only the most general hypotheses.
3. After all the examples are processed, output S and G.

Step-by-step Procedure

1. Define the helper functions covers(h, x), more_general(h1, h2) and show(S, G).
2. Read enjoysport.csv using pandas and separate the attributes (examples) and the target (labels) as lists.
3. Define candidate_elimination(examples, labels): initialise S with the first example and G with the most general hypothesis.
4. For a positive example, keep only the members of G that cover it and change the attributes of S that do not match to "?".
5. For a negative example, specialise every member of G that covers it using the values of S and keep only the most general hypotheses.
6. Print S and G after every example and display the final S and G.

Formulae

Consistent(h, D) $\Leftrightarrow$ h(x) = c(x) for every training example <x, c(x)> in D

Version space VS = { h $\in$ H | Consistent(h, D) }

S = the most specific members of VS ; G = the most general members of VS

Every hypothesis h with $S \leq h \leq G$ belongs to the version space

Python Program

ex-2a.py - D:/ex-2a.py (3.10.0)

File Edit Format Run Options Window Help

```
# Candidate-Elimination Algorithm - EnjoySport data (using pandas)
import pandas as pd

def covers(h, x):
    for i in range(len(h)):
        if h[i] != "?" and h[i] != x[i]:
            return False
    return True

def more_general(h1, h2):
    # True if h1 is more general than h2
    for i in range(len(h1)):
        if h1[i] != "?" and h1[i] != h2[i]:
            return False
    return True

def show(S, G):
    print("S :", S)
    print("G :")
    for g in G:
        print("   ", g)

def candidate_elimination(examples, labels):
    n = len(examples[0])
    S = list(examples[0])
    G = [["?"] * n]
```

ex-2a.py - D:/ex-2a.py (3.10.0)

File Edit Format Run Options Window Help

```
    for k in range(len(examples)):
        x = examples[k]
        if labels[k] == "Yes":
            # Positive example : prune G and generalise S
            G = [g for g in G if covers(g, x)]
            for j in range(n):
                if S[j] != x[j]:
                    S[j] = "?"
        else:
            # Negative example : specialise G using S
            new_G = []
            for g in G:
                if not covers(g, x):
                    new_G.append(g)
                else:
                    for j in range(n):
                        if g[j] == "?" and S[j] != "?" and S[j] != x[j]:
                            h = list(g)
                            h[j] = S[j]
                            new_G.append(h)
            # keep only the most general hypotheses
            G = []
            for g in new_G:
                general = [h for h in new_G if h != g and more_general(h, g)]
                if g not in G and len(general) == 0:
                    G.append(g)
```

ex-2a.py - D:/ex-2a.py (3.10.0)

File Edit Format Run Options Window Help

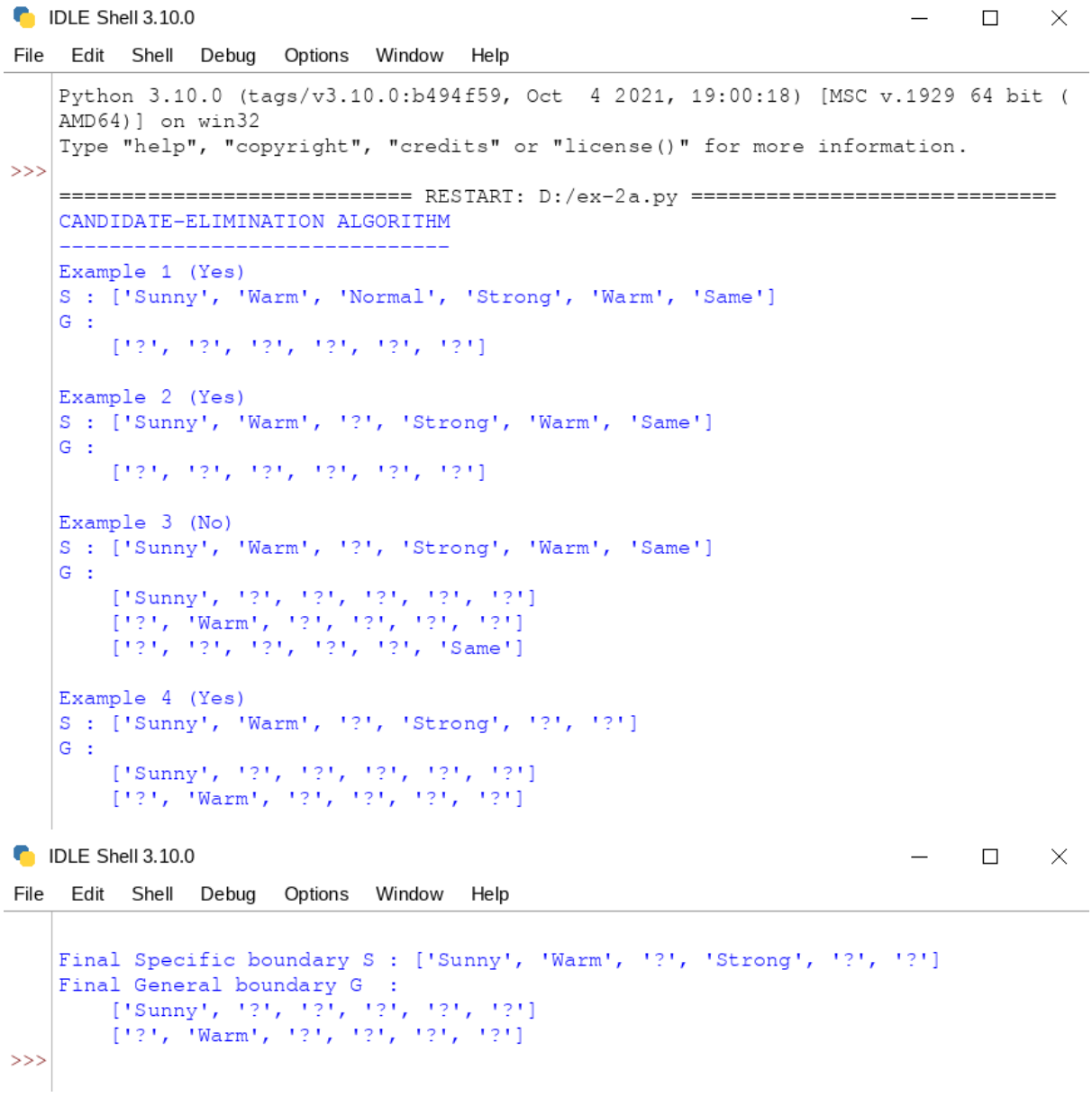
```
        print("Example", k + 1, "(" + labels[k] + ")")
        show(S, G)
        print()
    return S, G

data = pd.read_csv("enjoysport.csv")
examples = data.iloc[:, :-1].values.tolist()
labels = data.iloc[:, -1].values.tolist()

print("CANDIDATE-ELIMINATION ALGORITHM")
print("-------------------------------")
S, G = candidate_elimination(examples, labels)

print("Final Specific boundary S :", S)
print("Final General boundary G  :")
for g in G:
    print("   ", g)
```

Output



Result / Conclusion

The Candidate-Elimination algorithm was implemented successfully on the EnjoySport data set. The final boundaries are S = <Sunny, Warm, ?, Strong, ?, ?> and G = {<Sunny, ?, ?, ?, ?, ?>, <?, Warm, ?, ?, ?, ?>}. All hypotheses between S and G are consistent with the training examples.

Experiment No. 2(b) Date:

Experiment Question

For a given set of training data examples stored in a .CSV file, implement and demonstrate the Candidate-Elimination algorithm in python to output a description of the set of all hypotheses consistent with the training examples. [Sub-program (b): Weather data set, using the csv module]

Aim

To implement the Candidate-Elimination algorithm in Python using the csv module, reading the training examples of the Weather data set, and to output the specific boundary S and the general boundary G that describe all the hypotheses consistent with the training examples.

Concept

The version space is the subset of hypotheses of H that are consistent with all the training examples. The Candidate-Elimination algorithm represents it by two boundaries: S, the set of maximally specific hypotheses, and G, the set of maximally general hypotheses. Every hypothesis lying between S and G is consistent with the data.

A positive example generalises S and removes the inconsistent members of G. A negative example specialises G and removes the inconsistent members of S. Unlike FIND-S, the algorithm uses both positive and negative examples.

Dataset Description

The Weather data set (weather.csv) has 5 training examples, 4 input attributes (Outlook, Temp, Humidity, Wind) and the target attribute Play (Yes / No) which tells whether the outdoor game can be played.

Outlook	Temp	Humidity	Wind	Play
Sunny	Warm	Normal	Weak	Yes
Sunny	Warm	High	Weak	Yes
Rainy	Cold	High	Strong	No
Sunny	Hot	High	Weak	Yes
Overcast	Warm	Normal	Weak	No

Algorithm

1. Initialise S to the first (positive) training example and G to the most general hypothesis <?, ?, ..., ?>.
2. For each training example d:
   - If d is a positive example: remove from G every hypothesis inconsistent with d, and generalise S (replace by "?" every attribute that differs from d).
   - If d is a negative example: every member g of G that covers d is replaced by its minimal specialisations that exclude d and are still more general than S; then keep only the most general hypotheses.

3. After all the examples are processed, output S and G.

Step-by-step Procedure

1. Define the helper functions covers(h, x), more_general(h1, h2) and show(S, G).
2. Read weather.csv using the csv module; store the attribute values in the list examples and the last column in the list labels.
3. Define candidate_elimination(examples, labels): initialise S with the first example and G with the most general hypothesis.
4. For a positive example, keep only the members of G that cover it and change the attributes of S that do not match to "?".
5. For a negative example, specialise every member of G that covers it using the values of S and keep only the most general hypotheses.
6. Print S and G after every example and display the final S and G.

Formulae

Consistent(h, D) $\Leftrightarrow$ h(x) = c(x) for every training example <x, c(x)> in D

Version space VS = { h $\in$ H | Consistent(h, D) }

S = the most specific members of VS ; G = the most general members of VS

Every hypothesis h with $S \leq h \leq G$ belongs to the version space

Python Program

ex-2b.py - D:/ex-2b.py (3.10.0)

File Edit Format Run Options Window Help

```
# Candidate-Elimination Algorithm - Weather data (using the csv module)
import csv

def covers(h, x):
    for i in range(len(h)):
        if h[i] != "?" and h[i] != x[i]:
            return False
    return True

def more_general(h1, h2):
    # True if h1 is more general than h2
    for i in range(len(h1)):
        if h1[i] != "?" and h1[i] != h2[i]:
            return False
    return True

def show(S, G):
    print("S :", S)
    print("G :")
    for g in G:
        print("   ", g)

def candidate_elimination(examples, labels):
    n = len(examples[0])
    S = list(examples[0])
    G = [["?"] * n]
```

ex-2b.py - D:/ex-2b.py (3.10.0)

File Edit Format Run Options Window Help

```
    for k in range(len(examples)):
        x = examples[k]
        if labels[k] == "Yes":
            # Positive example : prune G and generalise S
            G = [g for g in G if covers(g, x)]
            for j in range(n):
                if S[j] != x[j]:
                    S[j] = "?"
        else:
            # Negative example : specialise G using S
            new_G = []
            for g in G:
                if not covers(g, x):
                    new_G.append(g)
                else:
                    for j in range(n):
                        if g[j] == "?" and S[j] != "?" and S[j] != x[j]:
                            h = list(g)
                            h[j] = S[j]
                            new_G.append(h)
            # keep only the most general hypotheses
            G = []
            for g in new_G:
                general = [h for h in new_G if h != g and more_general(h, g)]
                if g not in G and len(general) == 0:
                    G.append(g)
```

ex-2b.py - D:/ex-2b.py (3.10.0)

File Edit Format Run Options Window Help

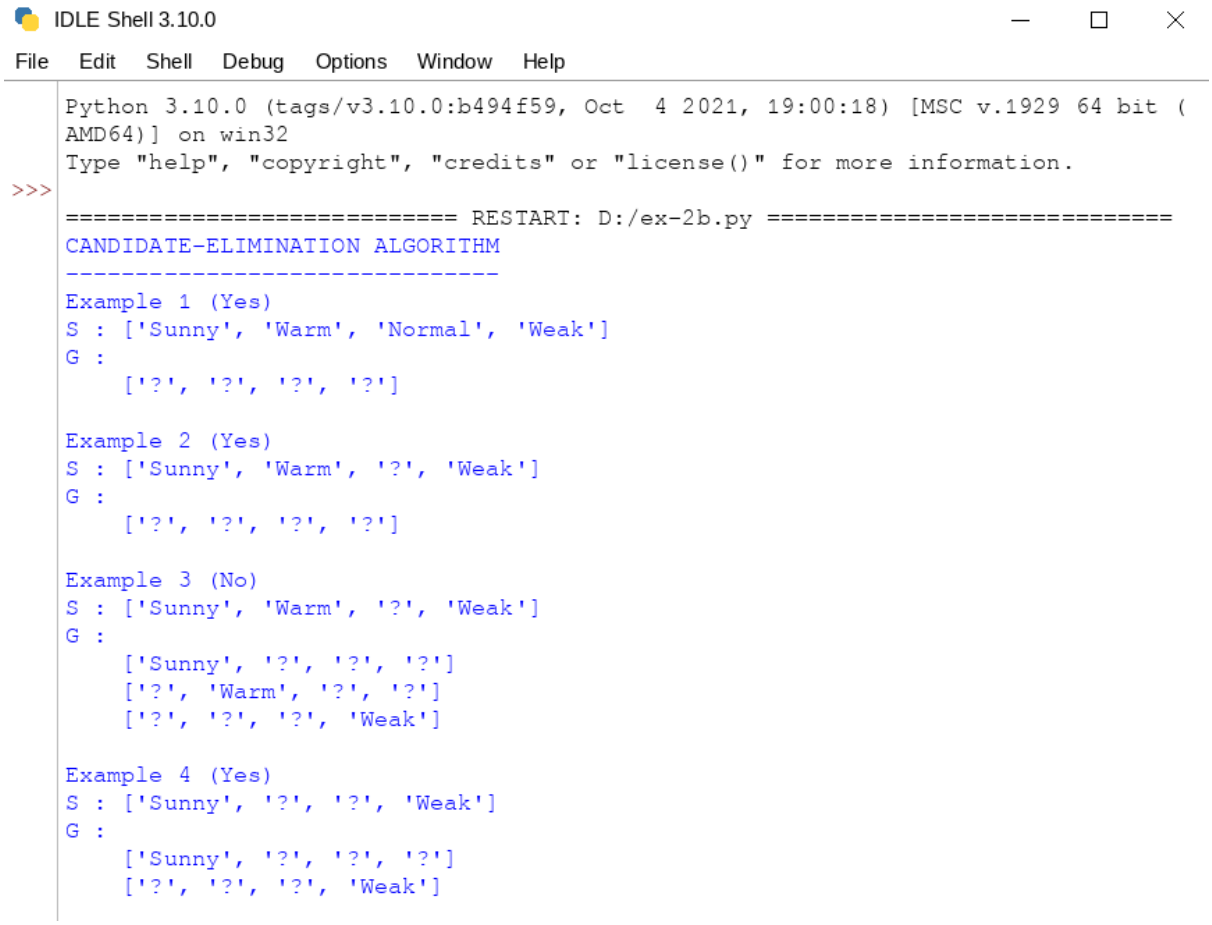
```
        print("Example", k + 1, "(" + labels[k] + ")")
        show(S, G)
        print()
    return S, G

examples = []
labels = []
with open("weather.csv") as f:
    reader = csv.reader(f)
    header = next(reader)
    for r in reader:
        examples.append(r[:-1])
        labels.append(r[-1])

print("CANDIDATE-ELIMINATION ALGORITHM")
print("-------------------------------")
S, G = candidate_elimination(examples, labels)

print("Final Specific boundary S :", S)
print("Final General boundary G  :")
for g in G:
    print("   ", g)
```

Output





Result / Conclusion

The Candidate-Elimination algorithm was implemented successfully on the Weather data set. The final boundaries are S = <Sunny, ?, ?, Weak> and G = {<Sunny, ?, ?, ?>}. The version space therefore contains the hypotheses <Sunny, ?, ?, ?> and <Sunny, ?, ?, Weak>.

Experiment No. 2(c) Date:

Experiment Question

For a given set of training data examples stored in a .CSV file, implement and demonstrate the Candidate-Elimination algorithm in python to output a description of the set of all hypotheses consistent with the training examples. [Sub-program (c): Mango Quality data set, using numpy]

Aim

To implement the Candidate-Elimination algorithm in Python using numpy, reading the training examples of the Mango Quality data set, and to output the specific boundary S and the general boundary G that describe all the hypotheses consistent with the training examples.

Concept

The version space is the subset of hypotheses of H that are consistent with all the training examples. The Candidate-Elimination algorithm represents it by two boundaries: S, the set of maximally specific hypotheses, and G, the set of maximally general hypotheses. Every hypothesis lying between S and G is consistent with the data.

A positive example generalises S and removes the inconsistent members of G. A negative example specialises G and removes the inconsistent members of S. Unlike FIND-S, the algorithm uses both positive and negative examples.

Dataset Description

The Mango Quality data set (mango.csv) has 5 training examples, 4 input attributes (Colour, Size, Smell, Firmness) and the target attribute GoodToBuy (Yes / No).

Colour	Size	Smell	Firmness	GoodToBuy
Yellow	Large	Sweet	Soft	Yes
Yellow	Medium	Sweet	Soft	Yes
Green	Large	Sour	Hard	No
Yellow	Large	Sweet	Firm	Yes
Green	Medium	Sweet	Soft	No

Algorithm

1. Initialise S to the first (positive) training example and G to the most general hypothesis <?, ?, ..., ?>.
2. For each training example d:
   - If d is a positive example: remove from G every hypothesis inconsistent with d, and generalise S (replace by "?" every attribute that differs from d).
   - If d is a negative example: every member g of G that covers d is replaced by its minimal specialisations that exclude d and are still more general than S; then keep only the most general hypotheses.
3. After all the examples are processed, output S and G.

Step-by-step Procedure

1. Define the helper functions covers(h, x), more_general(h1, h2) and show(S, G).
2. Load mango.csv using np.loadtxt() as a string array and convert the attributes and the target into the lists examples and labels.
3. Define candidate_elimination(examples, labels): initialise S with the first example and G with the most general hypothesis.
4. For a positive example, keep only the members of G that cover it and change the attributes of S that do not match to "?".
5. For a negative example, specialise every member of G that covers it using the values of S and keep only the most general hypotheses.
6. Print S and G after every example and display the final S and G.

Formulae

Consistent(h, D) $\Leftrightarrow$ h(x) = c(x) for every training example <x, c(x)> in D

Version space VS = { h $\in$ H | Consistent(h, D) }

S = the most specific members of VS ; G = the most general members of VS

Every hypothesis h with $S \le h \le G$ belongs to the version space

Python Program

ex-2c.py - D:/ex-2c.py (3.10.0)

File Edit Format Run Options Window Help

```
# Candidate-Elimination Algorithm - Mango Quality data (using numpy)
import numpy as np

def covers(h, x):
    for i in range(len(h)):
        if h[i] != "?" and h[i] != x[i]:
            return False
    return True

def more_general(h1, h2):
    # True if h1 is more general than h2
    for i in range(len(h1)):
        if h1[i] != "?" and h1[i] != h2[i]:
            return False
    return True

def show(S, G):
    print("S :", S)
    print("G :")
    for g in G:
        print("   ", g)

def candidate_elimination(examples, labels):
    n = len(examples[0])
    S = list(examples[0])
    G = [["?"] * n]
```

ex-2c.py - D:/ex-2c.py (3.10.0)

File Edit Format Run Options Window Help

```
    for k in range(len(examples)):
        x = examples[k]
        if labels[k] == "Yes":
            # Positive example : prune G and generalise S
            G = [g for g in G if covers(g, x)]
            for j in range(n):
                if S[j] != x[j]:
                    S[j] = "?"
        else:
            # Negative example : specialise G using S
            new_G = []
            for g in G:
                if not covers(g, x):
                    new_G.append(g)
                else:
                    for j in range(n):
                        if g[j] == "?" and S[j] != "?" and S[j] != x[j]:
                            h = list(g)
                            h[j] = S[j]
                            new_G.append(h)
            # keep only the most general hypotheses
            G = []
            for g in new_G:
                general = [h for h in new_G if h != g and more_general(h, g)]
                if g not in G and len(general) == 0:
                    G.append(g)
```

ex-2c.py - D:/ex-2c.py (3.10.0)

File Edit Format Run Options Window Help

```
        print("Example", k + 1, "(" + labels[k] + ")")
        show(S, G)
        print()
    return S, G

data = np.loadtxt("mango.csv", delimiter=",", dtype=str, skiprows=1)
examples = data[:, :-1].tolist()
labels = data[:, -1].tolist()

print("CANDIDATE-ELIMINATION ALGORITHM")
print("-------------------------------")
S, G = candidate_elimination(examples, labels)

print("Final Specific boundary S :", S)
print("Final General boundary G  :")
for g in G:
    print("   ", g)
```

Output





Result / Conclusion

The Candidate-Elimination algorithm was implemented successfully on the Mango Quality data set. The final boundaries are S = <Yellow, ?, Sweet, ?> and G = {<Yellow, ?, ?, ?>}. The version space contains the hypotheses <Yellow, ?, ?, ?>, <Yellow, ?, Sweet, ?> and no others.

Experiment No. 2(d) Date:

Experiment Question

For a given set of training data examples stored in a .CSV file, implement and demonstrate the Candidate-Elimination algorithm in python to output a description of the set of all hypotheses consistent with the training examples. [Sub-program (d): Loan Approval data set, using a function and Python lists]

Aim

To implement the Candidate-Elimination algorithm in Python using a function and Python lists for the Loan Approval data set, and to output the specific boundary S and the general boundary G that describe all the hypotheses consistent with the training examples.

Concept

The version space is the subset of hypotheses of H that are consistent with all the training examples. The Candidate-Elimination algorithm represents it by two boundaries: S, the set of maximally specific hypotheses, and G, the set of maximally general hypotheses. Every hypothesis lying between S and G is consistent with the data.

A positive example generalises S and removes the inconsistent members of G. A negative example specialises G and removes the inconsistent members of S. Unlike FIND-S, the algorithm uses both positive and negative examples.

Dataset Description

The Loan Approval data set has 5 training examples, 4 input attributes (Income, Credit, Employment, Property) and the target attribute Approved (Yes / No). The data is stored directly in a Python list in the program.

Income	Credit	Employment	Property	Approved
High	Good	Salaried	Owned	Yes
High	Good	Salaried	Rented	Yes
Low	Poor	Self-employed	Rented	No
High	Good	Self-employed	Owned	Yes
Low	Good	Salaried	Owned	No

Algorithm

1. Initialise S to the first (positive) training example and G to the most general hypothesis <?, ?, ..., ?>.
2. For each training example d:
   - If d is a positive example: remove from G every hypothesis inconsistent with d, and generalise S (replace by "?" every attribute that differs from d).
   - If d is a negative example: every member g of G that covers d is replaced by its minimal specialisations that exclude d and are still more general than S; then keep only the most general hypotheses.

3. After all the examples are processed, output S and G.

Step-by-step Procedure

1. Define the helper functions covers(h, x), more_general(h1, h2) and show(S, G).
2. Store the loan examples and their labels directly in the Python lists examples and labels.
3. Define candidate_elimination(examples, labels): initialise S with the first example and G with the most general hypothesis.
4. For a positive example, keep only the members of G that cover it and change the attributes of S that do not match to "?".
5. For a negative example, specialise every member of G that covers it using the values of S and keep only the most general hypotheses.
6. Print S and G after every example and display the final S and G.

Formulae

Consistent(h, D) $\Leftrightarrow$ h(x) = c(x) for every training example <x, c(x)> in D

Version space VS = { h $\in$ H | Consistent(h, D) }

S = the most specific members of VS ; G = the most general members of VS

Every hypothesis h with $S \leq h \leq G$ belongs to the version space

Python Program

ex-2d.py - D:/ex-2d.py (3.10.0)

File Edit Format Run Options Window Help

```
# Candidate-Elimination Algorithm - Loan Approval data (using lists)

def covers(h, x):
    for i in range(len(h)):
        if h[i] != "?" and h[i] != x[i]:
            return False
    return True

def more_general(h1, h2):
    # True if h1 is more general than h2
    for i in range(len(h1)):
        if h1[i] != "?" and h1[i] != h2[i]:
            return False
    return True

def show(S, G):
    print("S :", S)
    print("G :")
    for g in G:
        print("   ", g)

def candidate_elimination(examples, labels):
    n = len(examples[0])
    S = list(examples[0])
    G = [["?"] * n]
```

ex-2d.py - D:/ex-2d.py (3.10.0)

File Edit Format Run Options Window Help

```
    for k in range(len(examples)):
        x = examples[k]
        if labels[k] == "Yes":
            # Positive example : prune G and generalise S
            G = [g for g in G if covers(g, x)]
            for j in range(n):
                if S[j] != x[j]:
                    S[j] = "?"
        else:
            # Negative example : specialise G using S
            new_G = []
            for g in G:
                if not covers(g, x):
                    new_G.append(g)
                else:
                    for j in range(n):
                        if g[j] == "?" and S[j] != "?" and S[j] != x[j]:
                            h = list(g)
                            h[j] = S[j]
                            new_G.append(h)
            # keep only the most general hypotheses
            G = []
            for g in new_G:
                general = [h for h in new_G if h != g and more_general(h, g)]
                if g not in G and len(general) == 0:
                    G.append(g)
```

ex-2d.py - D:/ex-2d.py (3.10.0)

File Edit Format Run Options Window Help

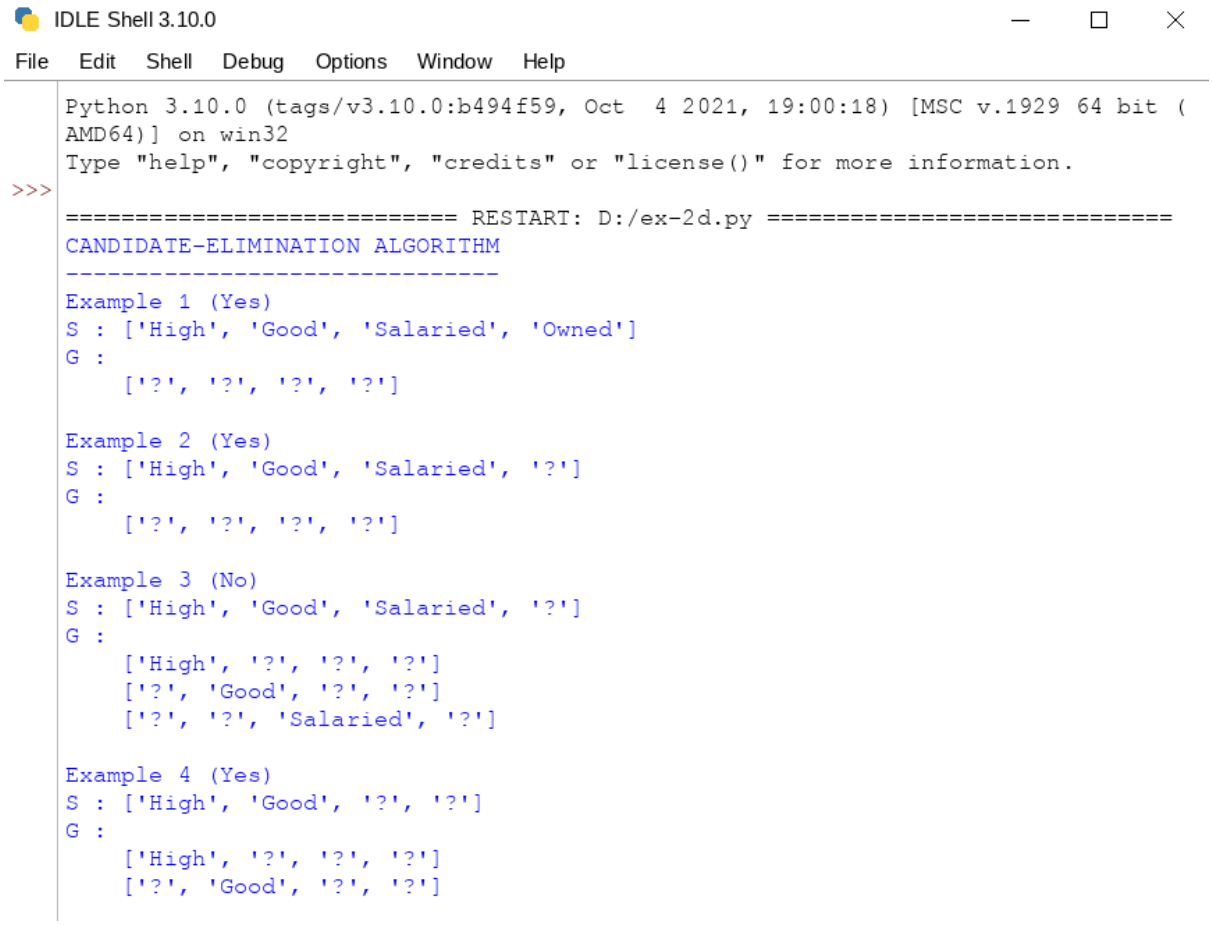
```
        print("Example", k + 1, "(" + labels[k] + ")")
        show(S, G)
        print()
    return S, G

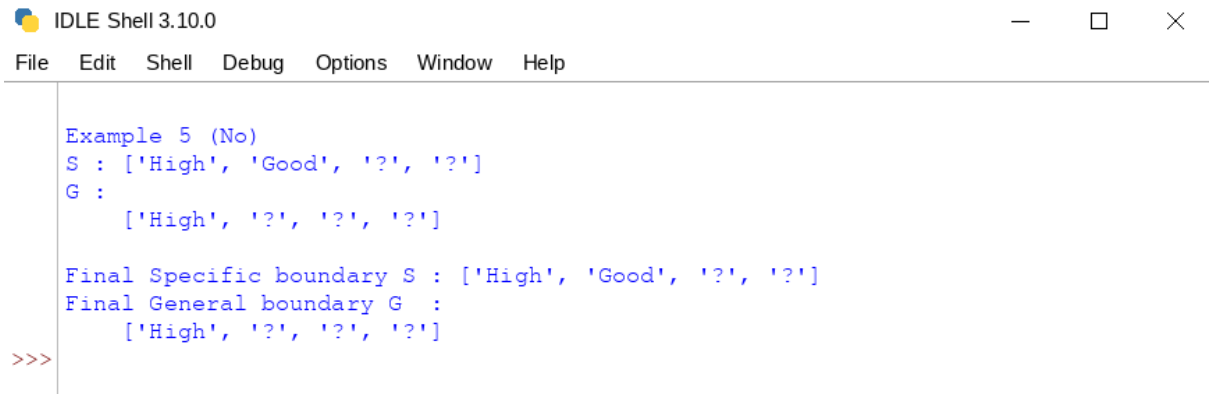
# Income, Credit, Employment, Property
examples = [["High", "Good", "Salaried", "Owned"],
            ["High", "Good", "Salaried", "Rented"],
            ["Low", "Poor", "Self-employed", "Rented"],
            ["High", "Good", "Self-employed", "Owned"],
            ["Low", "Good", "Salaried", "Owned"]]
labels = ["Yes", "Yes", "No", "Yes", "No"]

print("CANDIDATE-ELIMINATION ALGORITHM")
print("------------------------------")
S, G = candidate_elimination(examples, labels)

print("Final Specific boundary S :", S)
print("Final General boundary G  :")
for g in G:
    print("   ", g)
```

Output





Result / Conclusion

The Candidate-Elimination algorithm was implemented successfully on the Loan Approval data set. The final boundaries are S = <High, Good, ?, ?> and G = {<High, ?, ?, ?>}. The version space contains the hypotheses <High, ?, ?, ?>, <High, Good, ?, ?> and no others.

Experiment No. 2(e) Date:

Experiment Question

For a given set of training data examples stored in a .CSV file, implement and demonstrate the Candidate-Elimination algorithm in python to output a description of the set of all hypotheses consistent with the training examples. [Sub-program (e): Flu Diagnosis data set, using a class]

Aim

To implement the Candidate-Elimination algorithm in Python for the Flu Diagnosis data set, to output the boundaries S and G, and to use the version space to classify new patients.

Concept

The version space is the subset of hypotheses of H that are consistent with all the training examples. The Candidate-Elimination algorithm represents it by two boundaries: S, the set of maximally specific hypotheses, and G, the set of maximally general hypotheses. Every hypothesis lying between S and G is consistent with the data.

A positive example generalises S and removes the inconsistent members of G. A negative example specialises G and removes the inconsistent members of S. Unlike FIND-S, the algorithm uses both positive and negative examples.

Dataset Description

The Flu Diagnosis data set (flu.csv) has 5 training examples, 4 input attributes (Fever, Cough, Headache, BodyPain) and the target attribute Flu (Yes / No).

Fever	Cough	Headache	BodyPain	Flu
High	Yes	Severe	Yes	Yes
High	Yes	Mild	Yes	Yes
Normal	No	Mild	No	No
High	Yes	Severe	No	Yes
Normal	Yes	Mild	Yes	No

Algorithm

1. Initialise S to the first (positive) training example and G to the most general hypothesis <?, ?, ..., ?>.
2. For each training example d:
   - If d is a positive example: remove from G every hypothesis inconsistent with d, and generalise S (replace by "?" every attribute that differs from d).
   - If d is a negative example: every member g of G that covers d is replaced by its minimal specialisations that exclude d and are still more general than S; then keep only the most general hypotheses.
3. After all the examples are processed, output S and G.

Step-by-step Procedure

1. Define the helper functions covers(h, x), more_general(h1, h2) and show(S, G).
2. Read flu.csv using pandas and separate the attributes (examples) and the target (labels) as lists.
3. Define candidate_elimination(examples, labels): initialise S with the first example and G with the most general hypothesis.
4. For a positive example, keep only the members of G that cover it and change the attributes of S that do not match to "?".
5. For a negative example, specialise every member of G that covers it using the values of S and keep only the most general hypotheses.
6. Print S and G after every example and display the final S and G.
7. Classify new patients: Yes if the sample is covered by S, No if it is covered by no member of G, otherwise Not sure.

Formulae

Consistent(h, D) $\Leftrightarrow$ h(x) = c(x) for every training example <x, c(x)> in D

Version space VS = { h $\in$ H | Consistent(h, D) }

S = the most specific members of VS ; G = the most general members of VS

Every hypothesis h with $S \leq h \leq G$ belongs to the version space

Python Program

ex-2e.py - D:/ex-2e.py (3.10.0)

File Edit Format Run Options Window Help

```
# Candidate-Elimination Algorithm - Flu Diagnosis data (using pandas)
import pandas as pd

def covers(h, x):
    for i in range(len(h)):
        if h[i] != "?" and h[i] != x[i]:
            return False
    return True

def more_general(h1, h2):
    # True if h1 is more general than h2
    for i in range(len(h1)):
        if h1[i] != "?" and h1[i] != h2[i]:
            return False
    return True

def show(S, G):
    print("S :", S)
    print("G :")
    for g in G:
        print("   ", g)

def candidate_elimination(examples, labels):
    n = len(examples[0])
    S = list(examples[0])
    G = [["?"] * n]
```

ex-2e.py - D:/ex-2e.py (3.10.0)

File Edit Format Run Options Window Help

```
    for k in range(len(examples)):
        x = examples[k]
        if labels[k] == "Yes":
            # Positive example : prune G and generalise S
            G = [g for g in G if covers(g, x)]
            for j in range(n):
                if S[j] != x[j]:
                    S[j] = "?"
        else:
            # Negative example : specialise G using S
            new_G = []
            for g in G:
                if not covers(g, x):
                    new_G.append(g)
                else:
                    for j in range(n):
                        if g[j] == "?" and S[j] != "?" and S[j] != x[j]:
                            h = list(g)
                            h[j] = S[j]
                            new_G.append(h)
            # keep only the most general hypotheses
            G = []
            for g in new_G:
                general = [h for h in new_G if h != g and more_general(h, g)]
                if g not in G and len(general) == 0:
                    G.append(g)
```

ex-2e.py - D:/ex-2e.py (3.10.0)

File Edit Format Run Options Window Help

```
        print("Example", k + 1, "(" + labels[k] + ")")
        show(S, G)
        print()
    return S, G

data = pd.read_csv("flu.csv")
examples = data.iloc[:, :-1].values.tolist()
labels = data.iloc[:, -1].values.tolist()

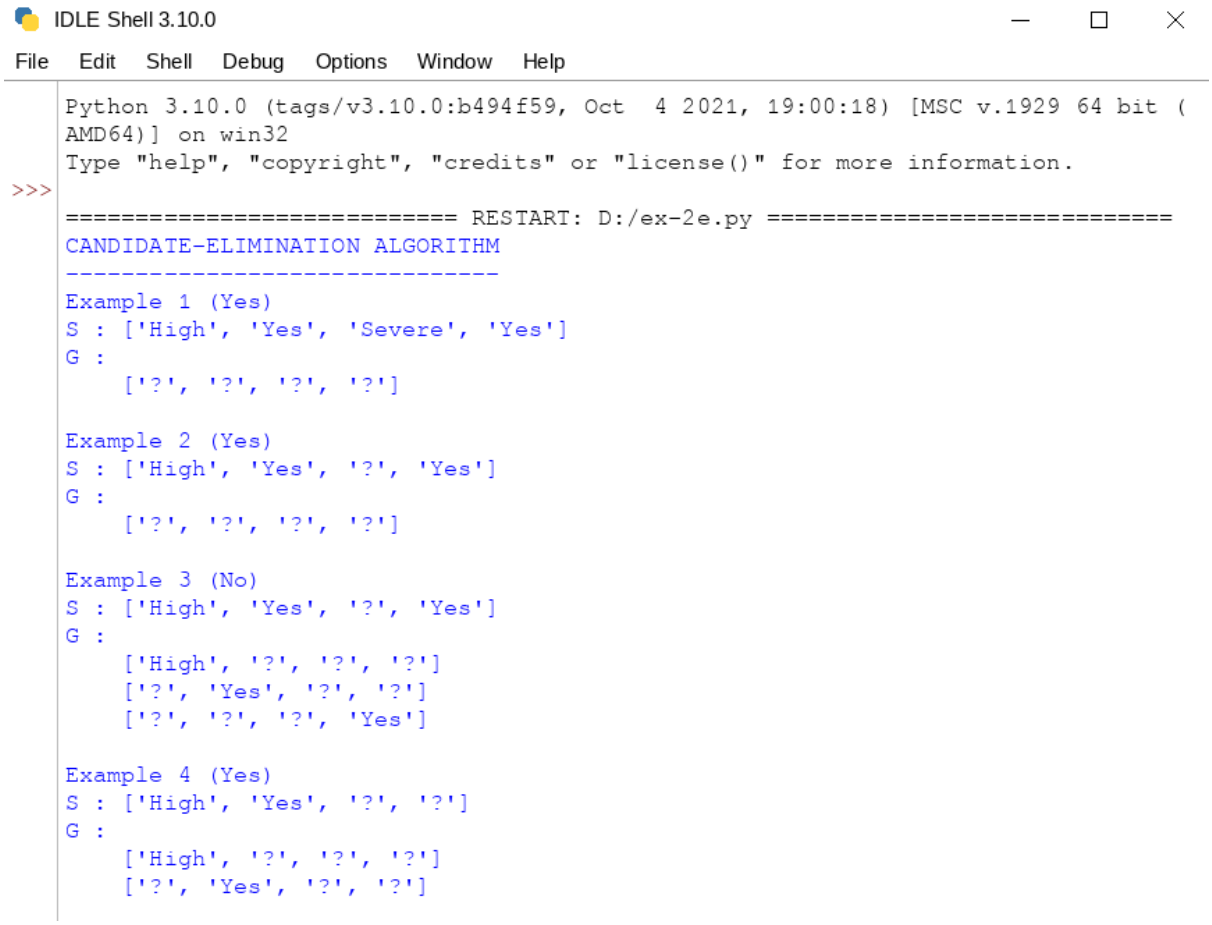
print("CANDIDATE-ELIMINATION ALGORITHM")
print("-------------------------------")
S, G = candidate_elimination(examples, labels)

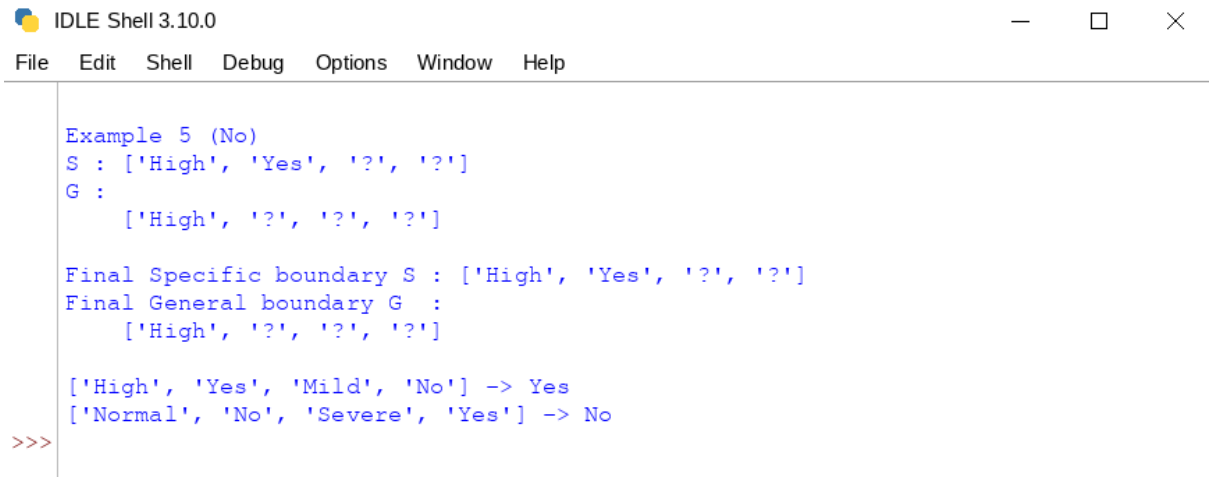
# Classify new patients using the version space
def classify(x):
    if covers(S, x):
        return "Yes"
    if not any(covers(g, x) for g in G):
        return "No"
    return "Not sure"

print("Final Specific boundary S :", S)
print("Final General boundary G  :")
for g in G:
    print("   ", g)

print()
for t in [["High", "Yes", "Mild", "No"], ["Normal", "No", "Severe", "Yes"]]:
    print(t, "->", classify(t))
```

Output





Result / Conclusion

The Candidate-Elimination algorithm was implemented successfully on the Flu Diagnosis data set. The final boundaries are S = <High, Yes, ?, ?> and G = {<High, ?, ?, ?>}. Using the version space, the patient (High, Yes, Mild, No) was classified as Yes and the patient (Normal, No, Severe, Yes) as No.